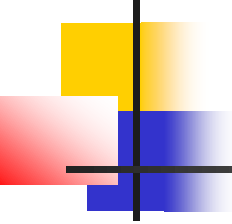


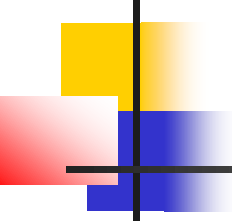
Communicating with Hardware

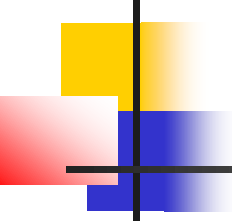


Topics

- Port-mapped vs. memory-mapped I/O
- Suppressing erroneous optimizations on I/O operations
- Kernel provided I/O helpers
- The parallel port
- The `short` example module

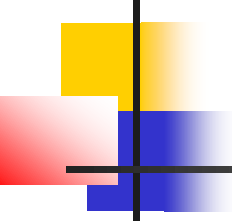
I/O Ports and I/O Memory

- 
- Peripheral devices are generally controlled by writing and reading its registers
 - At the hardware level, there is no conceptual difference between memory regions and I/O regions: both of them are accessed by asserting electrical signals on the address bus and control bus (i.e., the read and write signals)* and by reading from or writing to the data bus.
 - Common mechanisms to access registers
 - Memory address space operations
 - Memory-mapped I/O
 - Memory operations (i.e., read/write)
 - I/O address space
 - Port-mapped I/O
 - Special instructions



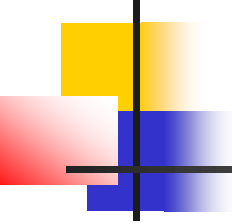
I/O Ports and I/O Memory

- Linux provides virtual I/O ports
- At the hardware level
 - Accessed at consecutive addresses
 - Assert commands to the address bus and control bus
 - Read from or write to the data bus



I/O Registers and Conventional Memory

- Need to watch out for CPU and compiler optimizations
 - I/O operations have side effects
 - Considerations when accessing registers
 - Caching
 - Automatically disabled by Linux initialization code
 - Read and write reordering
 - May need to insert memory barrier calls
 - Disable compiler optimizations



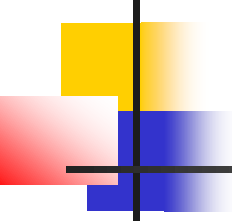
I/O Registers and Conventional Memory

- To prevent **compiler** optimizations across the barrier, call

```
#include <linux/compiler.h>
```

```
void barrier(void) ;
```

- Invalidate values in registers
- Forces a refetch
 - Compiler unaware when device changes values
- Suppresses instruction reordering across barrier
- Hardware is free to do its own reordering



I/O Registers and Conventional Memory

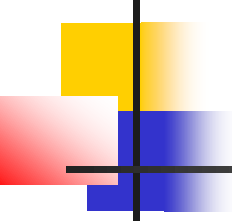
■ Other barrier calls

```
#include <asm/system.h>
/* all reads are completed before this barrier */
void rmb(void);

/* blocks reordering of reads (across the barrier) that
   depend on data from other reads
   http://lwn.net/Articles/5159/
*/
void read_barrier_depends(void);

/* all writes are completed before this barrier */
void wmb(void);

/* all reads & writes are completed before this barrier */
void mb(void);
```



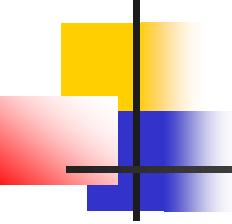
I/O Registers and Conventional Memory

- A typical usage

```
iowrite32(dev->registers.addr, io_destination_address);  
iowrite32(dev->registers.size, io_size);  
iowrite32(dev->registers.operation, DEV_READ);  
wmb();  
iowrite32(dev->registers.control, DEV_GO);
```

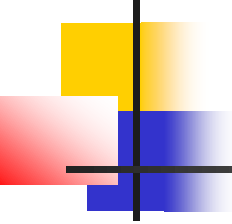
- Different barrier calls for SMP

```
void smp_rmb(void);  
void smp_read_barrier_depends(void);  
void smp_wmb(void);  
void smp_mb(void);
```

I/O Registers and Conventional Memory

- Most synchronization primitives function as memory barriers
 - spinlock, atomic_t

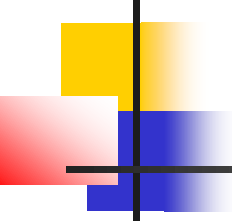


Using I/O Ports

- Allow drivers communicate with devices
- To allocate, call

```
#include <linux/ioport.h>
struct resource *request_region(unsigned long first,
                                unsigned long n,
                                const char *name);
```

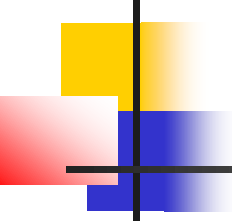
- Allocate **n** ports with **first**
- **name** is the name of the device
- Returns non-NULL on success



Using I/O Ports

- See `/proc/ioports` to see the current allocation

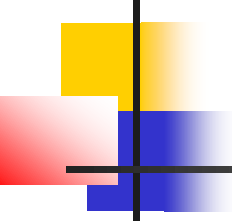
```
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0070-0077 : rtc
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
```



Using I/O Ports

- If your allocation fails
 - Try other ports
 - Remove the device module using those ports
- To free I/O ports, call

```
void release_region(unsigned long start, unsigned long n);
```



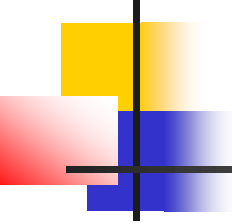
Manipulating I/O Ports

- Main interactions: reads and writes
- Needs to differentiate 8-bit, 16-bit, 32-bit ports

```
#include <asm/io.h>

/* 8-bit functions */
unsigned inb(unsigned port);
void outb(unsigned char byte, unsigned port);

/* 16-bit functions */
unsigned inw(unsigned port);
void outw(unsigned short word, unsigned port);
```



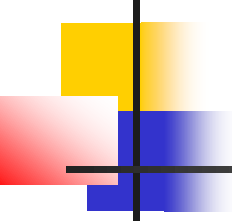
Manipulating I/O Ports

```
/* 32-bit functions */
```

```
unsigned inl(unsigned port);
```

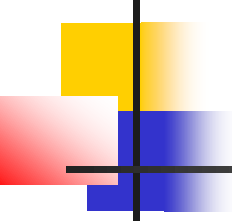
```
void outl(unsigned longword, unsigned port);
```

I/O Port Access from User Space

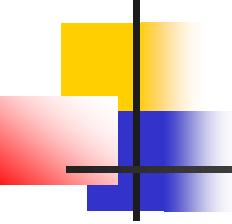


- Via `/dev/port`
- `#include <sys/io.h>`
- Same `inb/outb`, `inw/outw`, `inl/outl` calls
 - `ioperm()`
 - Sets the port access permission bits for the calling thread
 - `iopl()`
 - Changes I/O privilege level of calling process
 - Run as `root`

I/O Port Access from User Space

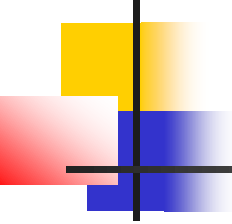


- See `misc-progs/inp.c` and `misc-progs/outp.c`
 - Create symlinks to the binary (binary performs differently depending on `argv[0]`)
 - `ln -s inb inp`
 - `ln -s inw inp`
 - `ln -s inl inp`
 - `ln -s outb outp`
 - `ln -s outw outp`
 - `ln -s outl outp`



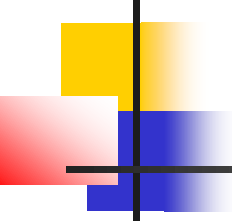
I/O Port Access from User Space

- Specify the port number to read and write
 - To read 1 byte from port 0x40
 - > `inb 40`
 - To write 1 byte "0xa5" to port 0x40
 - > `outb 40 1 a5`
 - Make sure you know what you are doing before executing these commands!
- `/dev/port` is a potential security hole



String Operations

- String instructions can transfer a sequence of bytes, words, or longs
- Available on some processors
- The port and the host system might have different byte ordering rules



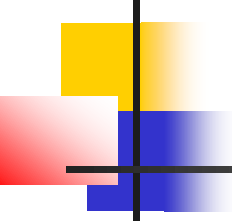
String Operations

■ Prototypes

```
void insb(unsigned port, void *addr, unsigned long count);  
void outsb(unsigned port, void *addr, unsigned long count);
```

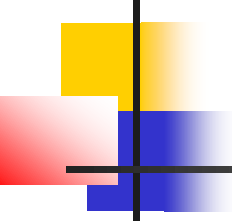
```
void insw(unsigned port, void *addr, unsigned long count);  
void outsw(unsigned port, void *addr, unsigned long count);
```

```
void insl(unsigned port, void *addr, unsigned long count);  
void outsl(unsigned port, void *addr, unsigned long count);
```



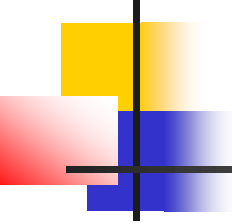
Pausing I/O

- Sometimes the CPU transfers data too quickly to or from the bus
 - Need to insert a small delay after each I/O instruction
 - Send `outb` to port `0x80` (on the x86)
 - Busy wait
 - See `<asm/io.h>` for details
 - Use pausing functions (e.g., `inb_p`, `outb_p`)



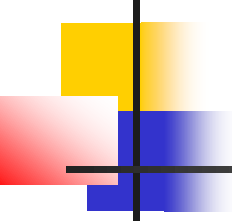
Platform Dependencies

- I/O instructions are highly CPU dependent by their nature
 - x86 and X86_64
 - `unsigned short` port numbers
 - ARM
 - Ports are memory-mapped
 - `unsigned int` port numbers



Platform Dependencies

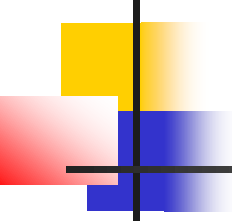
- MIPS and MIPS64
 - `unsigned long` port numbers
- PowerPC
 - `unsigned char *` ports on 32-bit systems
 - `unsigned long` on 64-bit systems
- SPARC
 - Memory-mapped I/O
 - `unsigned long` ports



An I/O Port Example

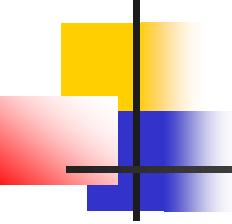
- A digital I/O port
 - Byte-wide I/O location
 - Either memory-mapped or port-mapped
 - Separate input pins and output pins (most of the time)
 - E.g., parallel port

An Overview of the Parallel Port



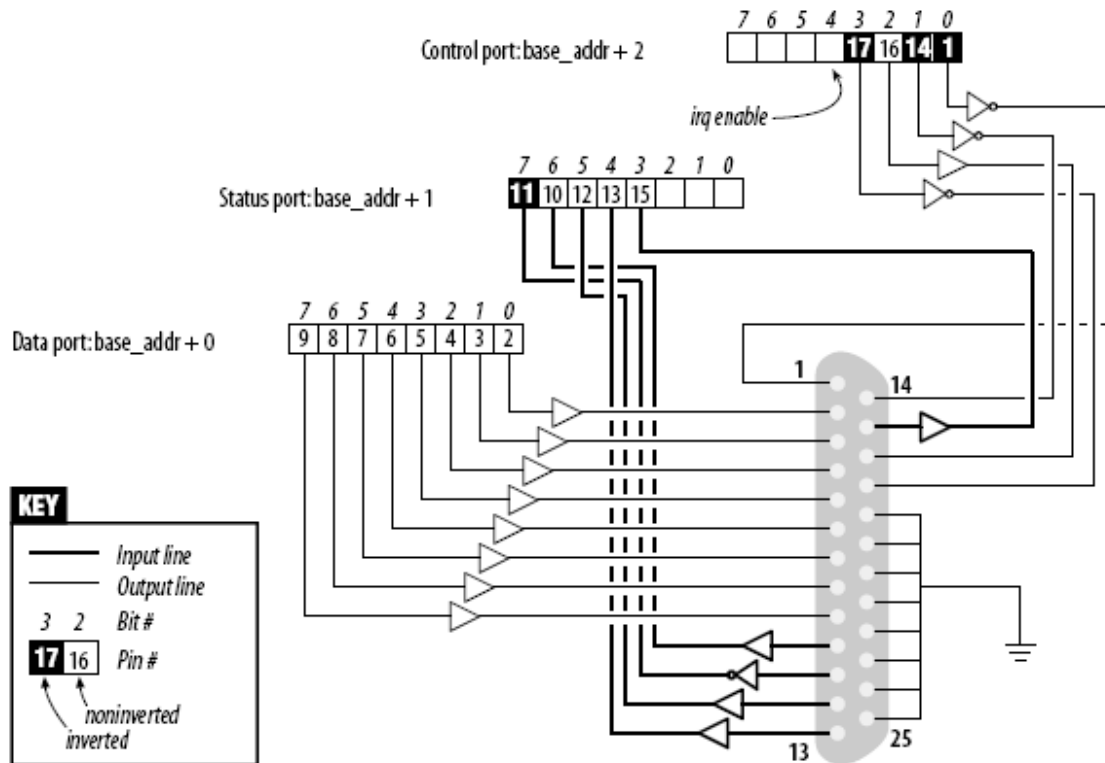
- 5V (TTL) logic levels
- Made up of three 8-bit ports
 - 12 output bits and 5 input bits
- First parallel interface consists of port 0x378-0x37a, second at 0x278-0x27a
 - First port (0x378/0x278) is a bidirectional data register
 - Pins 2-9

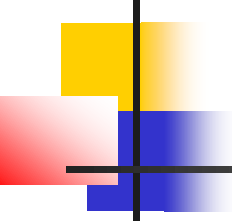
An Overview of the Parallel Port



- Second port is a status register
 - Online, out of paper, busy
- Third port is an output-only control register
 - Controls whether interrupts are enabled

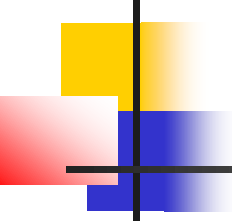
An Overview of the Parallel Port





A Sample Driver

- **short** (Simple Hardware Operations and Raw Tests)
 - Uses ports `0x378–0x37f`
 - `/dev/short0` reads and writes the 8-bit port `0x378`
 - `/dev/short1` reads and writes port `0x379...`
 - Not sophisticated enough to handle printers



A Sample Driver

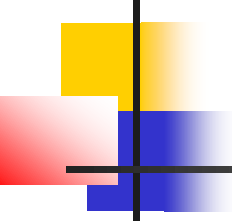
- `/dev/short0` is based on a tight loop

```
while (count--) {  
    outb(*(ptr++), port);  
    wmb(); /* write memory barrier */  
}
```

- To test, try

```
% echo -n "any string" > /dev/short0
```

- The last character stays on the output pins
- `-n` removes automatic insertion of `"\n"`



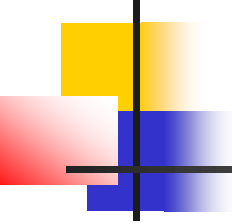
A Sample Driver

- To read, try

```
% dd if=/dev/short0 bs=1 count=1 | od -t x1
1+0 records in
1+0 records out
1 byte (1 B) copied, 4.4e-5 seconds, 22.7 kB/s
0000000 67
0000001
```

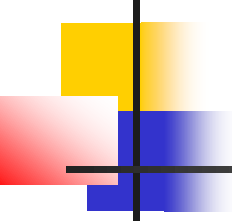
“g” in hex

- **dd** converts and copies a file
 - **bs** = transfer granularity in bytes
 - **count** = number of transfers
- **od** performs an octal dump
 - **-t x1** prints 1 byte in hex



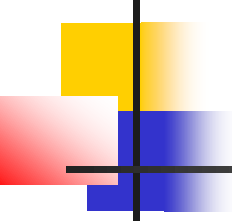
A Sample Driver

- Variants of `short`
 - `/dev/short0p` and the others use `outb_p` and `inb_p` pause functions
 - `/dev/short0s` and the others use the string instructions



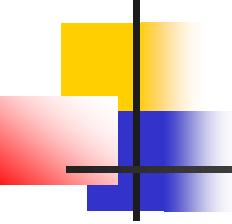
Using I/O Memory

- Outside of the x86 world, the main mechanism used to communicate with devices is through memory-mapped I/Os



Using I/O Memory

- Should not use pointers directly
 - Use wrappers to improve portability
- Depending on the platform
 - I/O memory may or may not be accessed through page tables
 - With the use of page tables, you need to call `ioremap` before doing any I/O
 - Without using the page tables, just use wrapper functions



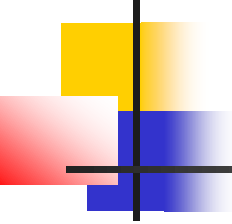
I/O Memory Allocation and Mapping

- To allocate I/O memory, call

```
#include <linux/ioport.h>
```

```
struct resource *request_mem_region(unsigned long start,  
                                   unsigned long len,  
                                   char *name);
```

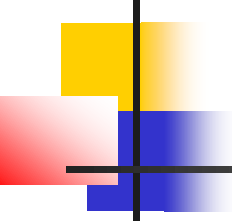
- **start**: starting memory location
- **len**: bytes
- **name**: displayed in `/proc/iomem`



I/O Memory Allocation and Mapping

■ more /proc/iomem

```
00000000-0009b7ff : System RAM
0009b800-0009ffff : reserved
000a0000-000bffff : Video RAM area
000c0000-000c7fff : Video ROM
000c8000-000c8fff : Adapter ROM
000f0000-000fffff : System ROM
00100000-7ff6ffff : System RAM
    00100000-002c7f2f : Kernel code
    002c7f30-003822ff : Kernel data
7ff70000-7ff77fff : ACPI Tables
7ff78000-7ff7ffff : ACPI Non-volatile Storage
...
```



I/O Memory Allocation and Mapping

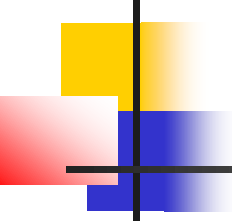
- To free memory regions, call

```
void release_mem_region(unsigned long start,  
                        unsigned long len);
```

- To make memory accessible, call

```
#include <asm/io.h>
```

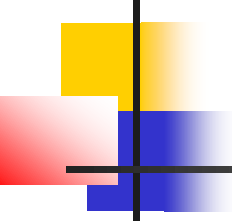
```
void *ioremap(unsigned long phys_addr, unsigned long size);  
void iounmap(void *addr);
```



Accessing I/O Memory

- Should use predefined macros to perform memory-mapped I/Os

```
unsigned int ioread8(void *addr);  
unsigned int ioread16(void *addr);  
unsigned int ioread32(void *addr);  
void iowrite8(u8 value, void *addr);  
void iowrite16(u16 value, void *addr);  
void iowrite32(u32 value, void *addr);
```

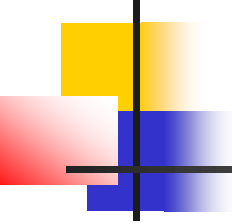


Accessing I/O Memory

- To perform repeated I/Os, use

```
void ioread8_rep(void *addr, void *buf, unsigned long count);  
void ioread16_rep(void *addr, void *buf, unsigned long count);  
void ioread32_rep(void *addr, void *buf, unsigned long count);  
void iowrite8_rep(void *addr, const void *buf,  
                  unsigned long count);  
void iowrite16_rep(void *addr, const void *buf,  
                   unsigned long count);  
void iowrite32_rep(void *addr, const void *buf,  
                   unsigned long count);
```

- **count**: number of repetitions



Accessing I/O Memory

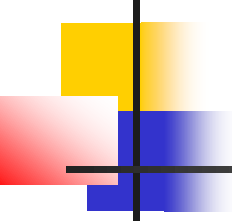
■ Other operations

```
void memset_io(void *addr, u8 value, unsigned int count);
```

```
void memcpy_fromio(void *dest, void *source,  
                  unsigned int count);
```

```
void memcpy_toio(void *dest, void *source,  
                unsigned int count);
```

■ count: in bytes

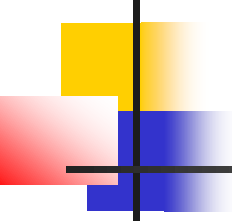


Ports as I/O Memory

- Linux 2.6 introduces `ioport_map`
 - Remaps I/O ports and makes them appear to be I/O memory

```
void *ioport_map(unsigned long port, unsigned int count);  
void ioport_unmap(void *addr);
```

- `port` = first port number
- `count` = number of I/O ports



Reusing short for I/O Memory

- To try the memory-mapped I/O, type

```
% ./short_load use_mem=1 base=0xb7ffffc0  
% echo -n 7 > /dev/short0
```

- The internal loop uses `iowrite8`

```
while (count--) {  
    iowrite8(*ptr++, address);  
    wmb( );  
}
```